

Day 6 – Detecting Expired Keys (Management Command)

Goals: Write a management command that marks expired keys and can be scheduled via cron.

Create the file structure:

```
games/  
  management/  
    __init__.py  
    commands/  
      __init__.py  
      check_expired_keys.py
```

games/management/commands/check_expired_keys.py:

```
from django.core.management.base import BaseCommand  
from django.utils import timezone  
from games.models import GameKey  
  
class Command(BaseCommand):  
    help = 'Mark active keys whose expiry has passed as expired.'  
  
    def handle(self, *args, **options):  
        expired_keys = GameKey.objects.filter(  
            expires_at__lte=timezone.now(),  
            status='active'  
        )  
        count = expired_keys.update(status='expired')  
        self.stdout.write(self.style.SUCCESS(f'Expired {count} keys.'))  
  
# Test manually  
python manage.py check_expired_keys
```

To schedule in production, add to crontab:

```
*/15 * * * * /path/to/.venv/bin/python /path/to/manage.py check_expired
```

Instructor Teaching Plan

Time Breakdown (90 min)

Segment	Duration	Activity
Management commands overview	15 min	What are management commands? When are they the right tool vs. a scheduled job vs. a cron script vs. a Celery task? Examples: migrate, collectstatic, createsuperuser. These are Django's built-in admin scripts — we can write our own.

File structure creation	10 min	Create <code>management/</code> , <code>management/__init__.py</code> , <code>management/commands/</code> , <code>management/commands/__init__.py</code> . Emphasize: both <code>__init__.py</code> files are required. Common mistake: missing one of them.
Write the command	25 min	Walk through <code>BaseCommand</code> , <code>help</code> , <code>handle()</code> , <code>self.stdout.write()</code> , <code>self.style.SUCCESS()</code> . Explain <code>.update()</code> as a bulk DB operation vs looping and calling <code>.save()</code> .
Test manually	15 min	Create a <code>GameKey</code> in admin with <code>expires_at</code> in the past. Run <code>python manage.py check_expired_keys</code> . Verify status changes in admin.
Cron scheduling	15 min	Show the cron syntax. Discuss: cron runs on one server — what if the server goes down? (Preview: Celery Beat solves this, but cron is fine for the bootcamp scope.)
Q&A + checkpoint	10 min	

🔑 Key Concepts to Introduce

- **BaseCommand** — the base class for all management commands. `handle()` is the entry point; `help` is the description shown in `manage.py help`.
- **`self.stdout.write()` vs `print()`** — management commands should use `self.stdout` so the output can be captured and redirected (e.g., in tests via `call_command()`).
- **`self.style.SUCCESS()` / `.ERROR()` / `.WARNING()`** — colorizes terminal output. Only applies when running in a terminal (not captured output).
- **Bulk update: `.update()`** — a single `UPDATE ... WHERE ...` SQL statement. Far more efficient than loading each object into Python and calling `.save()`. Trade-off: `save()` triggers model signals; `.update()` does not.
- **Timezone-aware datetimes** — `timezone.now()` returns a timezone-aware datetime. If `expires_at` were timezone-naive, the `__lte` comparison would fail. Always use `timezone.now()` in Django.
- **`expires_at__lte=timezone.now()`** — Django ORM field lookup: "less than or equal to now", i.e., "already expired".

⚠️ Common Mistakes to Address

- **Missing `__init__.py` files** — Django won't discover the command. The error is cryptic (Unknown command: 'check_expired_keys'). Teach students to check the directory structure first.
- **Using `.save()` in a loop** — works but generates N SQL queries. For 1000 expired keys, that's 1000 UPDATE statements vs one. Make the performance case.
- **Timezone-naive comparisons** — if a student uses `datetime.now()` (from the `datetime` module, not Django's `timezone`), they get a `TypeError` when comparing with a timezone-aware `expires_at`.
- **The queryset-after-update problem** — after `expired_keys.update(status='expired')`, the queryset is "stale" — the objects in Python still have `status='active'`. If you iterate after `.update()`, you get the pre-update values. (This will be critical in Day 8.)

? Check for Understanding

"Why do we use `self.stdout.write()` instead of `print()` inside a management command?"

Answer: Using `self.stdout.write()` is best practice because it integrates with Django's internal logging systems and output redirection. In testing, it allows the output to be intercepted and asserted on (using `call_command`), whereas `print()` writes directly to the standard system `stdout`, making it harder to test or suppress.

"What's the performance difference between `.update()` and looping + `.save()`? Are there situations where you'd prefer `.save()` anyway?"

Answer: `.update()` executes a single SQL `UPDATE` statement in the database, which is extremely fast and efficient for bulk operations. Looping and calling `.save()` executes a separate SQL query for each record (N queries), which is slow. However, you would prefer `.save()` if you need to trigger model `save()` overrides, pre/post-save signals, or validation, which `.update()` bypasses.

"What does `expires_at__lte=timezone.now()` translate to in SQL?"

Answer: It translates to a `WHERE` clause: `WHERE expires_at <= 'CURRENT_TIMESTAMP_VALUE'`. This filters for records where the expiration datetime is less than or equal to the current time.

"If we want this command to run every 15 minutes automatically, what are our options?"

Answer: 1) Setup a system-level cron job that executes the command through the `virtualenv python`. 2) Use a Celery Beat task that runs the management command (or its logic) on a schedule. 3) Use cloud-specific schedulers (like `Render Cron Jobs` or `AWS EventBridge`) to trigger the command.

✓ Day Checkpoint

Student creates a `GameKey` in `admin` with `expires_at` set to one hour ago and `status='active'`. They run `python manage.py check_expired_keys` and see the success message. Refreshing `admin` shows `status='expired'`.